

label pairs, as it processes the source code for the switch. We can do so, using an additional translation that accumulates just the pairs. Sketch a syntax-direction definition that produces the list of pairs, while also emitting code for the statements S_i that are the actions for each case.

6.9 Intermediate Code for Procedures

Procedures and their implementation will be discussed at length in Chapter 7, along with the run-time management of storage for names. We use the term *function* in this section for a procedure that returns a value. We briefly discuss function declarations and three-address code for function calls. In three-address code, a function call is unraveled into the evaluation of parameters in preparation for a call, followed by the call itself. For simplicity, we assume that parameters are passed by value; parameter-passing methods are discussed in Section 1.6.6.

Example 6.25: Suppose that a is an array of integers, and that f is a function from integers to integers. Then, the assignment

$$n = f(a[i]);$$

might translate into the following three-address code:

```

1)  t1 = i * 4
2)  t2 = a [ t1 ]
3)  param t2
4)  t3 = call f, 1
5)  n = t3

```

The first two lines compute the value of the expression $a[i]$ into temporary t_2 , as discussed in Section 6.4. Line 3 makes t_2 an actual parameter for the call on line 4 of f with one parameter. Line 5 assigns the value returned by the function call to t_3 . Line 6 assigns the returned value to n . $\square$

The productions in Fig. 6.52 allow function definitions and function calls. (The syntax generates unwanted commas after the last parameter, but is good enough for illustrating translation.) Nonterminals D and T generate declarations and types, respectively, as in Section 6.3. A function definition generated by D consists of keyword **define**, a return type, the function name, formal parameters in parentheses and a function body consisting of a statement. Nonterminal F generates zero or more formal parameters, where a formal parameter consists of a type followed by an identifier. Nonterminals S and E generate statements and expressions, respectively. The production for S adds a statement that returns the value of an expression. The production for E adds function calls, with actual parameters generated by A . An actual parameter is an expression.

$$\begin{aligned}
D &\rightarrow \textbf{define } T \textbf{ id } (F) \{ S \} \\
F &\rightarrow \epsilon \mid T \textbf{ id } , F \\
S &\rightarrow \textbf{return } E ; \\
E &\rightarrow \textbf{id } (A) \\
A &\rightarrow \epsilon \mid E , A
\end{aligned}$$

Figure 6.52: Adding functions to the source language

Function definitions and function calls can be translated using concepts that have already been introduced in this chapter.

- *Function types.* The type of a function must encode the return type and the types of the formal parameters. Let *void* be a special type that represents no parameter or no return type. The type of a function *pop()* that returns an integer is therefore “function from *void* to *integer*.” Function types can be represented by using a constructor *fun* applied to the return type and an ordered list of types for the parameters.
- *Symbol tables.* Let *s* be the top symbol table when the function definition is reached. The function name is entered into *s* for use in the rest of the program. The formal parameters of a function can be handled in analogy with field names in a record (see Fig. 6.18). In the production for *D*, after seeing **define** and the function name, we push *s* and set up a new symbol table

$$Env.push(top); \quad top = \textbf{new } Env(top);$$

Call the new symbol table, *t*. Note that *top* is passed as a parameter in **new** *Env(top)*, so the new symbol table *t* can be linked to the previous one, *s*. The new table *t* is used to translate the function body. We revert to the previous symbol table *s* after the function body is translated.

- *Type checking.* Within expressions, a function is treated like any other operator. The discussion of type checking in Section 6.5.2 therefore carries over, including the rules for coercions. For example, if *f* is a function with a parameter of type *real*, then the integer 2 is coerced to a *real* in the call *f*(2).
- *Function calls.* When generating three-address instructions for a function call **id**(*E*, *E*, . . . , *E*), it is sufficient to generate the three-address instructions for evaluating or reducing the parameters *E* to addresses, followed by a **param** instruction for each parameter. If we do not want to mix the parameter-evaluating instructions with the *param* instructions, the attribute *E.addr* for each expression *E* can be saved in a data structure

such as a queue. Once all the expressions are translated, the `param` instructions can be generated as the queue is emptied.

The procedure is such an important and frequently used programming construct that it is imperative for a compiler to good code for procedure calls and returns. The run-time routines that handle procedure parameter passing, calls, and returns are part of the run-time support package. Mechanisms for run-time support are discussed in Chapter 7.

6.10 Summary of Chapter 6

The techniques in this chapter can be combined to build a simple compiler front end, like the one in Appendix A. The front end can be built incrementally:

- ◆ *Pick an intermediate representation:* An intermediate representation is typically some combination of a graphical notation and three-address code. As in syntax trees, a node in a graphical notation represents a construct; the children of a node represent its subconstructs. Three address code takes its name from instructions of the form $x = y \text{ op } z$, with at most one operator per instruction. There are additional instructions for control flow.
- ◆ *Translate expressions:* Expressions with built-up operations can be unwound into a sequence of individual operations by attaching actions to each production of the form $E \rightarrow E_1 \text{ op } E_2$. The action either creates a node for E with the nodes for E_1 and E_2 as children, or it generates a three-address instruction that applies `op` to the addresses for E_1 and E_2 and puts the result into a new temporary name, which becomes the address for E .
- ◆ *Check types:* The type of an expression $E_1 \text{ op } E_2$ is determined by the operator `op` and the types of E_1 and E_2 . A coercion is an implicit type conversion, such as from *integer* to *float*. Intermediate code contains explicit type conversions to ensure an exact match between operand types and the types expected by an operator.
- ◆ *Use a symbol table to implement declarations:* A declaration specifies the type of a name. The width of a type is the amount of storage needed for a name with that type. Using widths, the relative address of a name at run time can be computed as an offset from the start of a data area. The type and relative address of a name are put into the symbol table due to a declaration, so the translator can subsequently get them when the name appears in an expression.
- ◆ *Flatten arrays:* For quick access, array elements are stored in consecutive locations. Arrays of arrays are flattened so they can be treated as a one-